國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 07

姓名：黃聖傑

學號：114310140

Lab07-Q1.Q2
Complete the following parts in the sample code: **- precedence** **- InfixToPostfix** **Add comments to the program to clearly describe its actions.**

Code

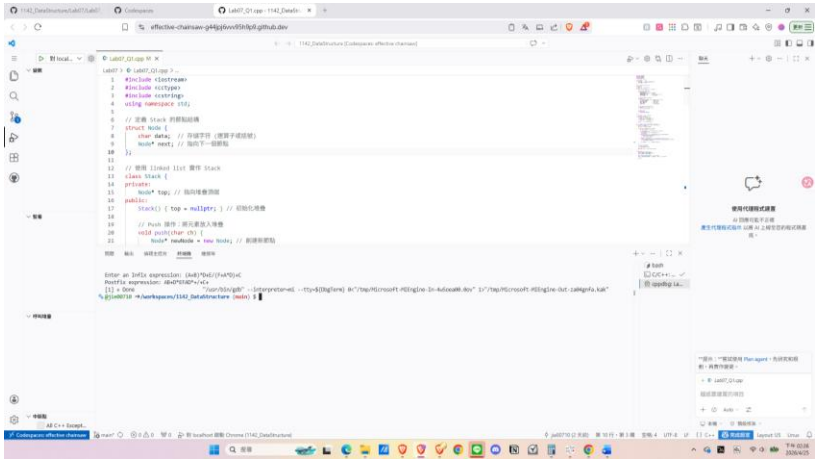
```
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;  // 存儲字符 (運算子或括號)
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push 操作：將元素放入堆疊
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;       // 存入數據
        newNode->next = top;      // 讓新節點指向目前的頂端
        top = newNode;            // 更新頂端指標
    }

    // Pop 操作：移除並回傳頂端元素
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
        char ch = top->data;        // 取得頂端元素
        Node* temp = top;           // 暫存頂端節點
```

```
        top = top->next;            // 移動頂端指標
        delete temp;                // 釋放記憶體
        return ch;                  // 回傳彈出的元素
    }

    // Peek 操作：取得頂端元素但不移除
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
    if (op == '*' || op == '/') {
        return 2; // 乘除的優先順序為 2
    } else if (op == '+' || op == '-') {
        return 1; // 加減的優先順序為 1
    }
    return -1; // 其他字符的優先順序為 -1
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    Stack s; // 創建堆疊
    int j = 0; // 後序表達式的索引

    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];// 讀取當前字符

        if (isalnum(ch)) { // 如果是 Operand，直接加入後序表達式
            postfix[j++] = ch;
        } else if (ch == '(') { // 左括號 push 進 stack
            s.push(ch);
```

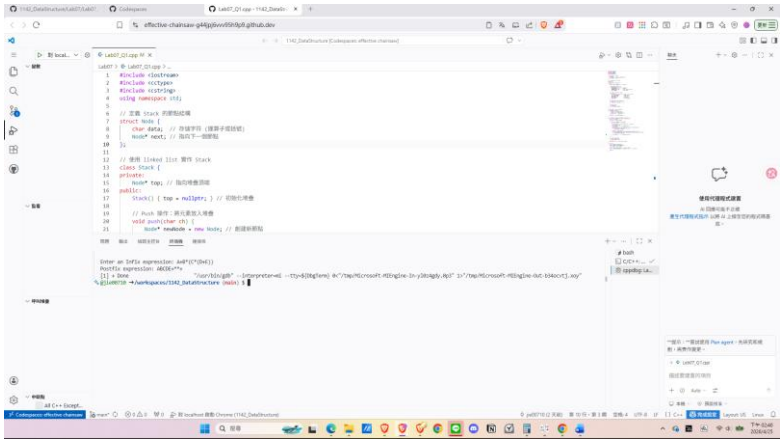
```
        } else if (ch == ')') { // 遇到右括號，pop 出 stack 中的元素直到左括號
            while (!s.isEmpty() && s.peek() != '(') {
                postfix[j++] = s.pop();
            }
            s.pop(); // pop 出左括號
        } else { // 遇到 operator，將 stack 中優先順序較高或相同的 operator
pop 出來
            while (!s.isEmpty() && precedence(s.peek()) >=
precedence(ch)) { // 比較順序
                postfix[j++] = s.pop();
            }
            s.push(ch); // 將當前讀取的元素 push 進 stack
        }
    }

    // 彈出堆疊中剩餘的元素
    while (!s.isEmpty()) {
        postfix[j++] = s.pop();
    }
    postfix[j] = '\0'; // 結束後序表達式字符串
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

    return 0;
}
```

Result

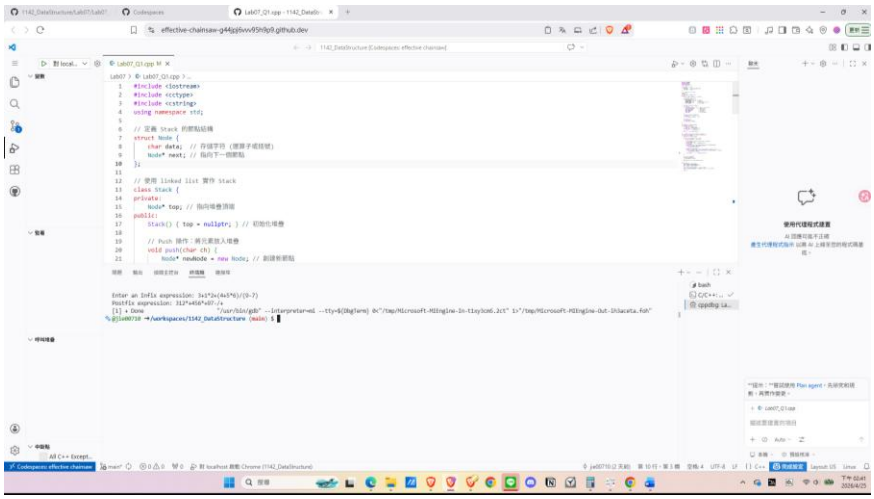
(a)



(b)



(c)



Lab07-Q3.

Draw a table showing the step-by-step process of converting each infix expression to postfix (following the method presented in the lecture notes.).

Discussion

(A+B)*D+E/(F+A*D)+C

Token	Stack					Output
	[0]	[1]	[2]	[3]	[4]	
(	(					
A	(					A
+	(	+				A
B	(	+				AB
)						AB+
*	*					AB+
D	*					AB+D
+	+					AB+D*
E	+					AB+D*E
/	+	/				AB+D*E
(	+	/	(			AB+D*E
F	+	/	(			AB+D*EF
+	+	/	(	+		AB+D*EF
A	+	/	(	+		AB+D*EFA
*	+	/	(	+	*	AB+D*EFA
D	+	/	(	+	*	AB+D*EFAD
)	+	/				AB+D*EFAD*+
+	+					AB+D*EFAD*+/+
C	+					AB+D*EFAD*+/+C
<eos>						AB+D*EFAD*+/+C+

A+B*(C*(D+E))

Token	Stack						Output
	[0]	[1]	[2]	[3]	[4]	[5]	
A							A
+	+						A
B	+						AB
*	+	*					AB
(	+	*	(				AB
C	+	*	(				ABC
*	+	*	(	*			ABC
(	+	*	(	*	(		ABC
D	+	*	(	*	(		ABCD
+	+	*	(	*	(	+	ABCD
E	+	*	(	*	(	+	ABCDE
)	+	*	(	*			ABCDE+
)	+	*					ABCDE+*
<eos>							ABCDE+**+

3+1*2+(4+5*6)/(9-7)

Token	Stack					Output
	[0]	[1]	[2]	[3]	[4]	
3						3
+	+					3
1	+					31
*	+	*				31
2	+	*				312
+	+					312*+
(	+	(				312*+
4	+	(				312*+4
+	+	(	+			312*+4
5	+	(	+			312*+45
*	+	(	+	*		312*+45
6	+	(	+	*		312*+456
)	+					312*+456*+
/	+	/				312*+456*+
(	+	/	(			312*+456*+
9	+	/	(			312*+456*+9
-	+	/	(	-		312*+456*+9
7	+	/	(	-		312*+456*+97
)	+	/				312*+456*+97-
<eos>						312*+456*+97-/+